# Harness Engineering架构

**讲师：肖斌**

## 直播间的同学，可以先找助教老师要一份文档

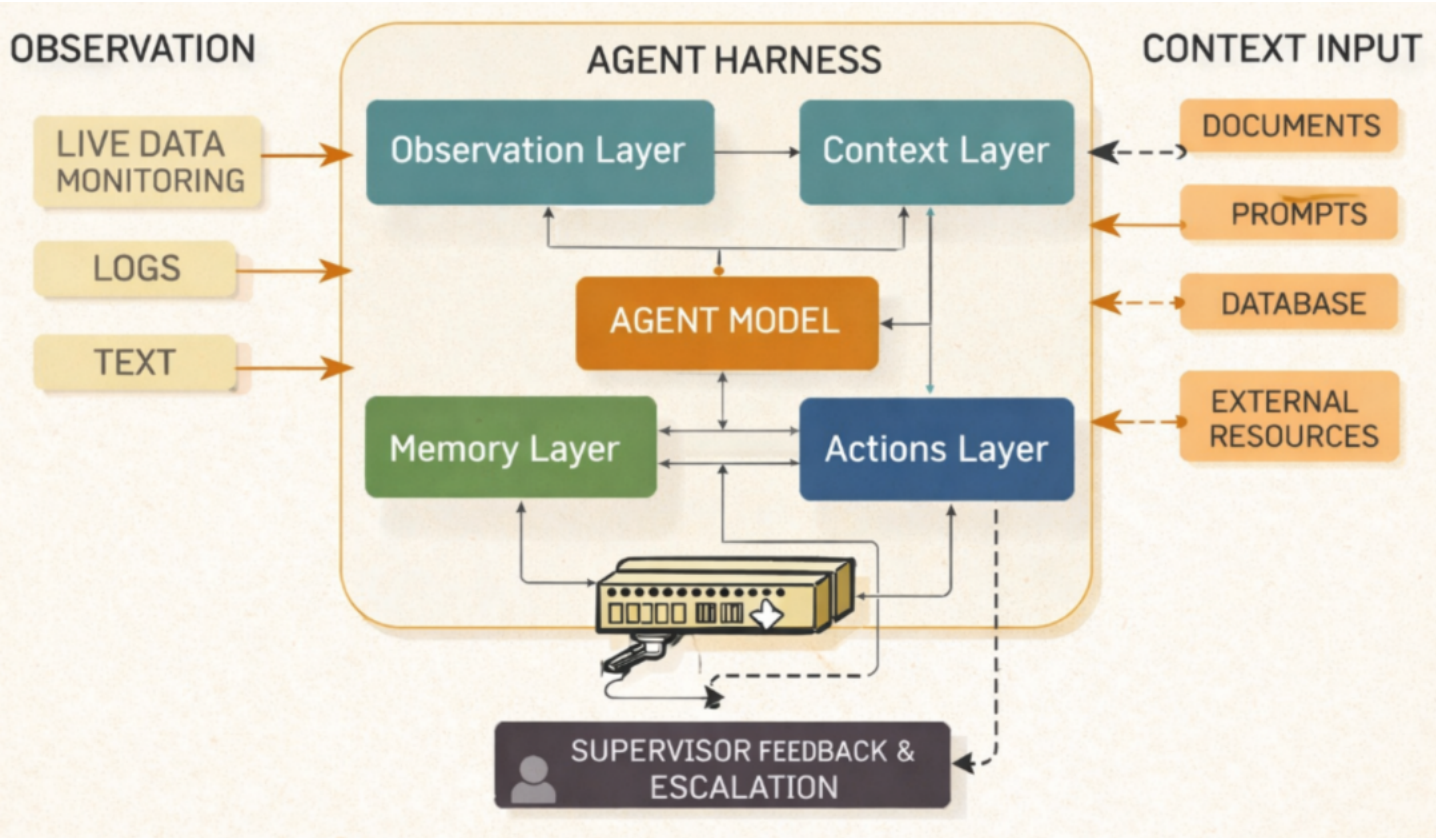
码士集团
www.mashibing.com

## 1、什么是 Harness Engineering?

· `Prompt Engineering`（2022-2024 主流）主要优化单次交互的质量，重点是 “一句提示词该怎么写，模型才更容易给出想要的结果”

· `Context Engineering`（2025 年2月后兴起）则动态构建知识、记忆、RAG，解决 “模型看什么” 的问题，减少幻觉、提高检索命中率

· `Harness Engineering`（当前阶段）重点构建整个运行环境，解决 “模型怎么把长链路任务稳定做完” 的系统级问题。




**Harness Engineering** 是当前 AI Agent 开发领域的一个核心范式转变。它不再仅仅关注如何通过提示词（Prompt Engineering）或上下文管理（Context Engineering）来“优化模型输出”，而是聚焦于

**构建包裹在模型之外的一整套系统化基础设施**，旨在将大语言模型（LLM）那种“不稳定、非确定性”的智能，转化为能够**稳定、可靠、长时间执行复杂任务**的工作引擎。

- **核心定义**：Harness（马具/缰绳）指的是除了模型本身之外的所有东西——工具、记忆、规划、安全护栏、执行循环、状态管理等。LangChain 团队将其精炼为公式：**Agent = Model + Harness**。模型提供“思考”能力，而 Harness 负责“让它真的能干活，而且别出大事”。
- **解决的问题**：传统 Agent 在应对长周期、多步骤的复杂任务时，常面临上下文窗口爆炸、状态丢失、工具调用混乱、缺乏规划、无法从失败中恢复等工程难题。Harness Engineering 正是为了解决这些“工程上的‘稳’的问题”而生的。

码士集团
www.mashibing.com

## 2、DeepAgents 中全面采用 Harness Engineering：

DeepAgents 框架本身就是 Harness Engineering 思想的**具体实现**。它通过一套预配置的、模块化的“能力套件”（即 Agent Harness），让开发者无需从零搭建复杂系统，只需通过配置即可获得一个功能完备的深度智能体。

### 模块一：规划能力（Planning）

- **功能**：提供一个 `write_todos` 工具，允许智能体维护一个结构化的任务清单。
- **作用**：帮助智能体跟踪多个任务及其状态（`‘pending’`，`‘in_progress’`，`‘completed’`），以组织复杂的多步骤工作，尤其适用于长周期任务。
- **Harness 价值**：将非结构化的模型思考，转化为可追踪、可恢复的确定性工作流。这避免了 Agent 在复杂任务中迷失方向，是长周期任务的基础。

### 模块二：虚拟文件系统（Virtual Filesystem）

- **功能**：提供一个可配置的虚拟文件系统，支持 `ls`（列出文件）、`read_file`（读取文件，支持图片和大型文件）、`write_file`（创建文件）、`edit_file`（编辑文件）、`glob`（模式匹配）、`grep`（内容搜索）和 `execute`（执行命令，仅在沙盒后端可用）等工具。
- **作用**：为其他能力（如技能、记忆、代码执行）提供基础存储和操作层，也可用于构建自定义工具。
- **Harness 价值**：这是对抗 **Context Rot（上下文腐烂）** 的核心手段。它通过外部存储扩展了有效的上下文窗口，显著降低了 Token 消耗，并让 Agent 能处理远超单次模型限制的长任务。

### 模块三：任务委托/子智能体（Task Delegation / Subagents）

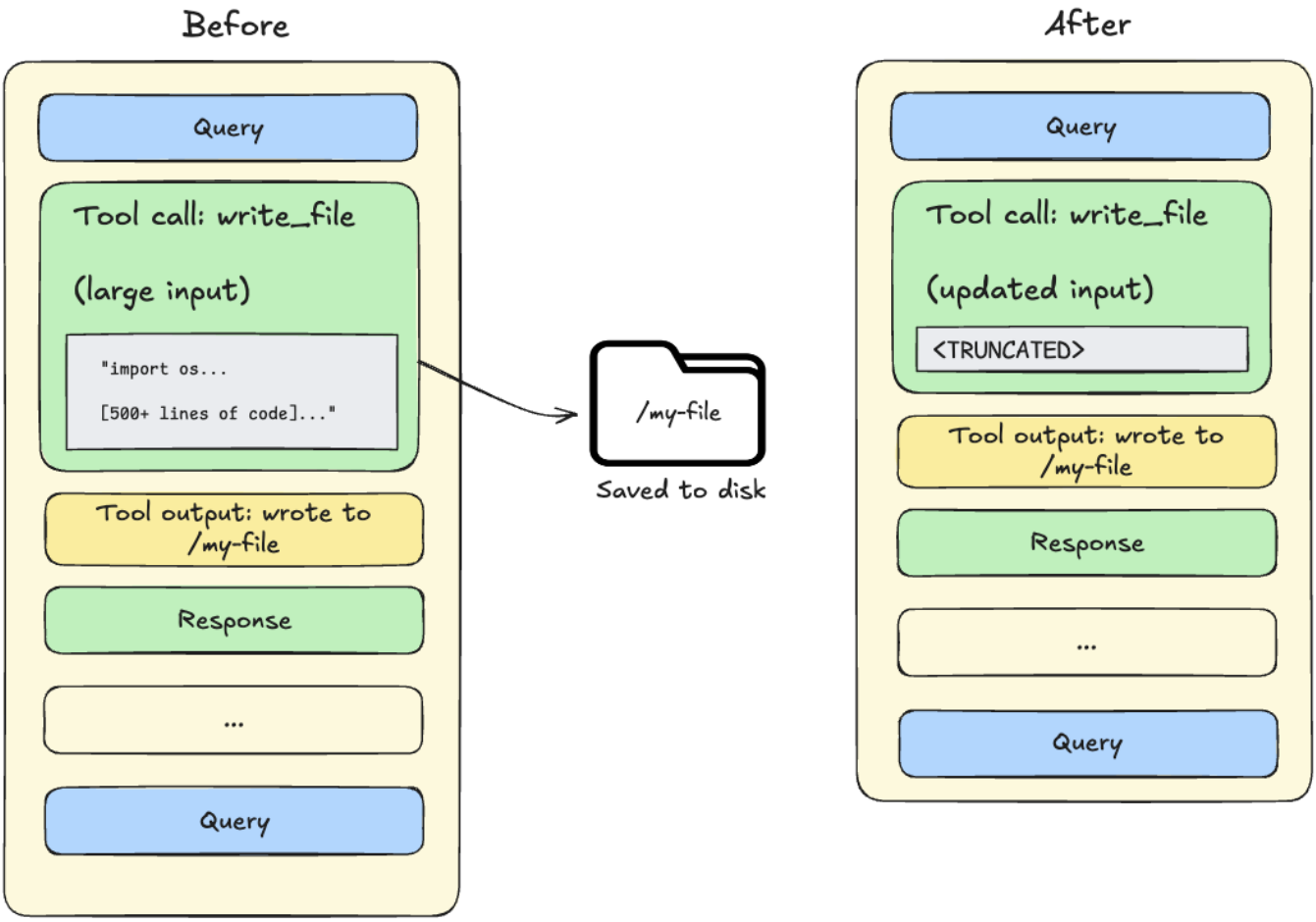
- **功能**：主 Agent 通过内置的 `task` 工具和 `SubAgentMiddleware`，可以将专项任务（如“代码审查”、“网络研究”）委托给一个独立的子Agent。开发者可以预定义具有不同系统提示和工具集的子Agent（通过 `subagents` 参数），实现专业化分工。
- **作用**：
  - **上下文隔离**：子智能体的工作不会干扰主智能体的上下文。
  - **并行执行**：多个子智能体可以并发运行。
  - **专业化**：可以为子智能体配置不同的工具集。
  - **Token高效**：将大型子任务的上下文压缩为单一结果报告返回给主智能体。
- **Harness 价值**：实现了**上下文隔离**和**算力分配**。子Agent在独立的上下文中运行，其繁杂的中间过程不会污染主Agent的思维空间，最终仅返回一个精简的结果。这本质上是为 Agent 系统引入了“多线程”和“微服务”架构。

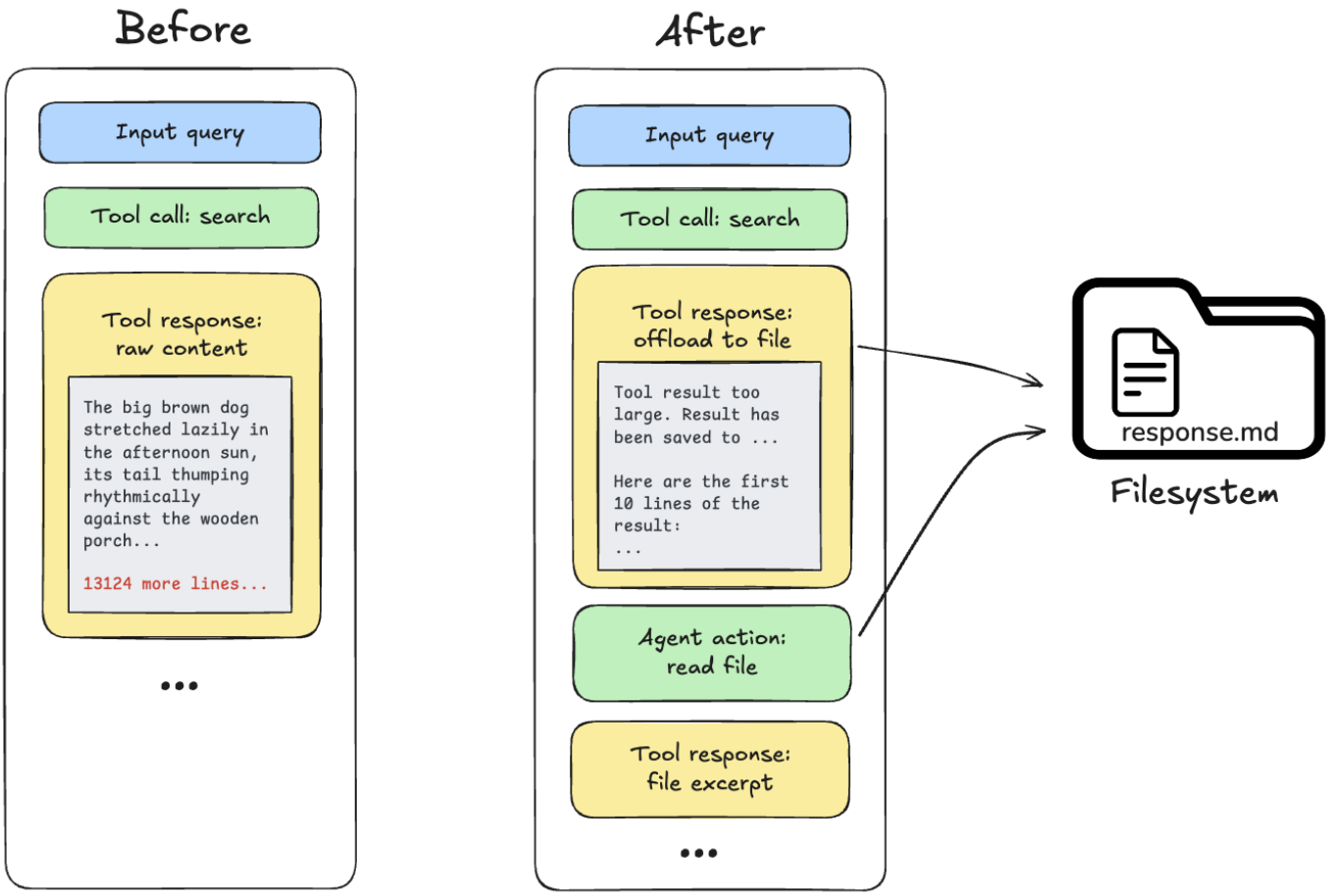
### 模块四：上下文管理（Context Management）

这是智能体处理长对话和复杂任务的关键，通过多种技术管理其工作记忆（上下文），确保不超出模型的处理窗口限制。主要包括：

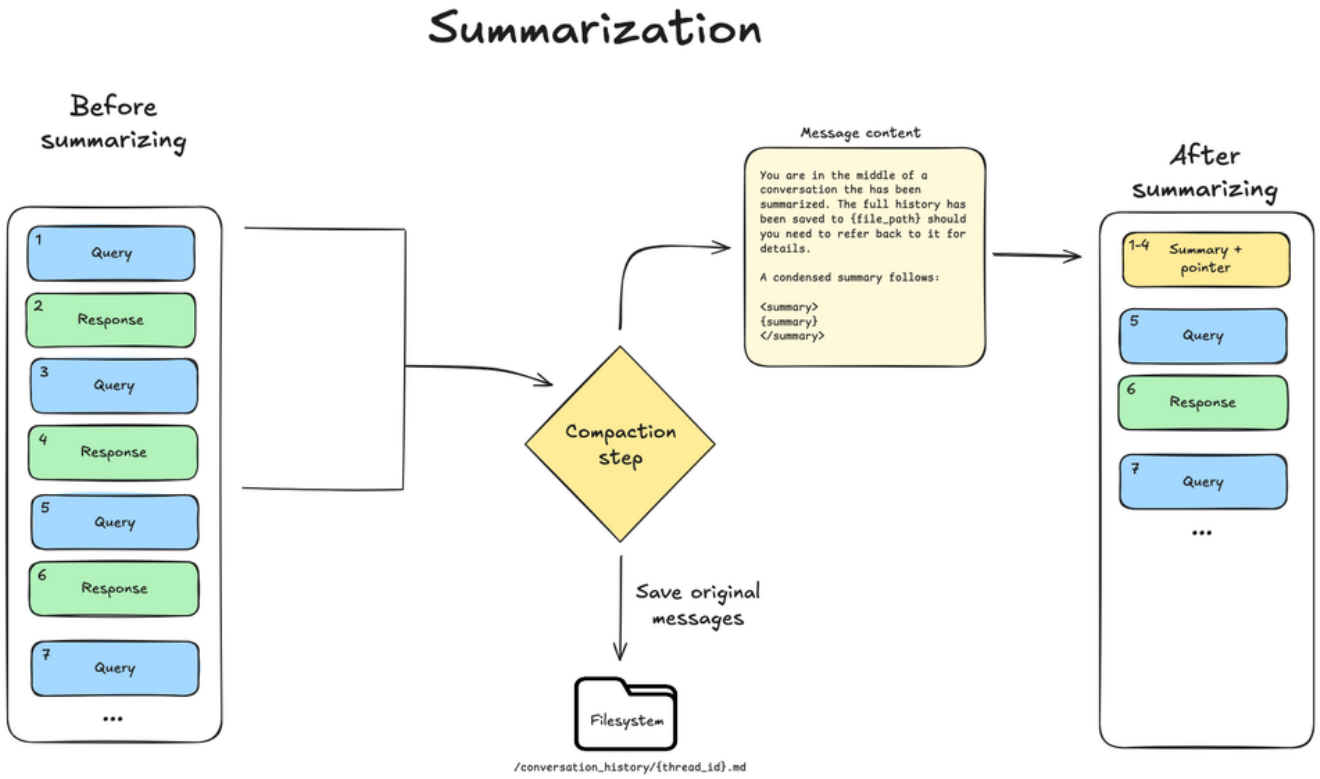
- **输入上下文**：包括系统提示词、待办列表、记忆、技能、文件系统工具说明等，在智能体启动时注入。
- **运行时上下文压缩**：
  - **卸载大型工具输入和结果**：当工具调用的输入或结果超过一定大小时（默认20，000 tokens），智能体会自动将其内容保存到文件系统，并在对话历史中用文件路径和摘要替换，以节省上下文空间。当会话上下文超过模型窗口的85%时，会触发此操作。

## Truncate stale tool inputs



- **摘要化 (Summarization)**：当上下文大小达到极限且无可卸载内容时，智能体会将过去的对话历史总结为一段结构化摘要，替换原有的详细历史，同时将完整历史保存到文件系统。




- **长期记忆 (Long-term memory)**：通过混合存储后端，允许智能体将特定路径（如 `/memories/`）下的信息持久化存储，使其能够跨不同的会话和线程访问，用于存储用户偏好、累积知识等。
- **Harness 价值**：注入领域知识与持久状态。

### 模块五：代码执行（Code Execution）与安全沙箱 (Sandbox)

- **功能**：当使用沙盒后端时，会向智能体暴露一个 `execute` 工具，允许其在隔离环境中运行 shell 命令。
- **作用**：使智能体能够安装依赖、运行脚本、执行代码，从而完成更复杂的任务，同时保证了主机系统的安全。
- **Harness 价值**：遵循 **“Trust the LLM”但隔离执行环境** 的安全哲学。它赋予 Agent 与真实操作系统交互的能力，同时通过沙箱机制防止破坏性操作，是编码、运维等场景的基石。

### 模块六：人工介入（Human-in-the-loop, HITL）

- **功能**：通过 `interrupt_on` 参数配置，可以在指定的工具调用前暂停智能体执行，等待人工批准或修改输入。

- **作用**：为具有破坏性或昂贵的操作（如编辑文件、调用API）增加安全闸门，便于交互式调试和指导。
- **Harness 价值**：在高风险或关键操作前插入确定性的人工控制点。这是将 Agent 集成到生产工作流中的必备安全机制。

### 模块七：技能包加载和管理（Skills）

- **功能**：采用“渐进式披露”，仅在需要时加载，以减少系统提示的 token 占用。适用于任务特定、内容可能很多的场景。
  - - **渐进式披露**：这是技能加载的核心机制。智能体启动时，**仅读取**所有 `SKILL.md` 文件的**元数据部分**。当接收到用户提示后，智能体根据元数据中的描述判断是否有相关技能可用。只有匹配到相关技能时，才会去**读取该技能文件夹的完整内容**（包括 `SKILL.md` 的详细指令和其他资源文件）。这大大减少了初始上下文的 token 消耗。
  - - **智能体使用流程**：匹配（根据描述）→ 读取（完整技能内容）→ 执行（遵循技能指令）。
- **作用**：Skills 是用于扩展 Deep Agent 能力的可重用模块。
- **Harness 价值**：实现了知识的模块化、按需加载。让 Agent 的行为具备领域专长。

码士集团
www.mashibing.com

## 3、DeepAgents框架中Harness Engineering 的核心原理

1. **配置优于编码**：DeepAgents 将上述所有复杂能力封装为可配置的**中间件（Middleware）** 栈。开发者通过 `create_deep_agent` 函数，以声明式的方式组合这些中间件，而非编写底层控制流。这实现了 **“Batteries Included, Customize in Minutes”**（开箱即用，分钟级定制）。
2. **中间件架构**：每个核心能力（规划、文件、子Agent等）都是一个独立的 `AgentMiddleware`。这些中间件在 Agent 运行的生命周期各阶段插入钩子（hooks），动态注入工具、修改提示、管理上下文。这种架构使得 Harness 高度模块化和可扩展。
3. **后端协议抽象**：统一的 `BackendProtocol` 抽象了文件存储和执行环境。无论是内存、本地磁盘、云存储还是沙箱，对 Agent 而言都是统一的 `ls`、`read`、`write` 接口。这种设计实现了存储与计算环境的解耦和灵活替换。
4. **上下文工程自动化**：Harness 承担了“上下文工程师”的角色。它自动执行摘要压缩、内容卸载、按需加载等策略，确保模型在任何时刻看到的都是最相关、最精炼的信息，从而将有限的模型注意力集中在核心决策上。
5. **从智能到系统**：Harness Engineering 的本质是将模型的“认知能力”系统化、工程化。它通过**规划**来管理目标，通过**文件系统**来管理状态，通过**子Agent**来管理复杂度，通过**安全沙箱**来管理风险，通过**人在回路**来管理不确定性。最终，它将一个“聪明的对话者”转变为一个“可靠的操作者”。

在 DeepAgents 框架中全面采用 **Harness Engineering**，意味着您的开发范式发生了根本转变：

- **从前**：您需要手动编写任务分解逻辑、管理上下文窗口、实现子Agent通信、构建安全机制。
- **现在**：您只需通过 `create_deep_agent` 配置所需的“能力模块”（规划、文件、子Agent、技能、内存、沙箱、人工审批），即可立即获得一个具备生产级可靠性的深度智能体。